



What is Semi-Supervised Learning (SSL)?



- Mix of labeled and unlabeled data
- Useful when labeled data is scarce but unlabeled data is plenty
- Bridges gap between supervised (labeled) and unsupervised (no labels) learning



Why Expectation-Maximization (EM)?

- Helps train models using both labeled data
- for unlabeled, guess labels using oncertain
- Iteratively improves the model to maximize likelihood of data



How EM Works (Step-by-Step)

- Initialize Model
- E-Step: For labeled data use known labels; for unlabeled, guess labels using model
- M-Step: Update model to fit both types
- Repeat until convergence



Evaluation

- Test final model on separate test data



Applications



Image classification (e.y. Google)

- NLP (e.g. Stanford NLP)
- Speech recognition



Challenges

- Assumes missing data is random
- Risk of local optima
- Slow for large datasets

Made by
Humna Imran



What is K-Nearest Neighbors (KNN)?



A simple algorithm that classifies data based on how close it is to others

Example: To guess if a new user will cancel a subscription, KNN checks how similar users behaved

Why is KNN called a Lazy Learner?

No training phase — it just stores data
Makes predictions only when needed by checking nearby data points

How Does KNN Work? (Steps)

- Choose K: "K" = number of neighbors

- Measure Distance:

- Euclidean (straight line)
- Mankattan (grid-based)
- Minkowski (generalized)

- Find Nearest Neighbors: sort points by distance

- Predict: majority vote for classification or average for regression

Made by
Humna
Imran



How to Choose the Best K?



→ Too small → overfitting



→ Too big → underfitting

Use odd values, cross-validation, elbow method



KNN for Classification vs Regression



Classification	Regression
Majority voting	→ Mean value
Class label	→ Numeric output



Applications

→ Recommendation Systems

- Netflix, Amazon

📧 Spam Detection

👤 Customer Segmentation

🗣️ Speech Recognition



Disadvantages



- Slow for large datasets

• Struggles with high-dimensional data

• Sensitive to noise and irrelevant features



Advantages

- Easy to understand and implement
- No training required
- Works for both classification & regression



Final Summary

- Easy to understand and implement
- No training required
- Works for both classification & regression



Made by Humna Imran



KNN vs Other Algorithms

Decision Trees	Better
Random Forests	Less
SVM	Powerful
Neural Nets	Large



Final Summary



KNN = classify by
 Great for small
 datasets and quick
 Choose k wisely
 Good for recommen

id filtering



What are Artificial Neural Networks (ANNs)?

- ✦ Inspired by the human brain: **billions of neurons**
- ANNs learn from data, not hand-coded rules

Structure of ANNs

Brain Part	ANN Equivalent
Dendrites	Inputs
Soma (cell body)	Node/Neuron
Synapses	Weights
Axon	Output
Synaptic plasticity	Backpropagation



Layers of ANN:

- ✦ Input Layer - receives
- ✦ Hidden Layers - process
- ✦ Output Layer - gives result

How ANN Learns (Training Process)

- ✦ Input Training Data (eg, cat images)
- ✦ Pattern Recognition (features like whiskers, ears)
- ✦ Testing & Validation

Types of Neural Networks

- Feedforward Neural Network
Input → Hidden → Output
- Convolutional Neural Network
(CNN) - image-focused,
- Recurrent Neural Network
(RNN) (for memory)

How ANN Learns (Training Process)



Recurrent Neural Network (RNN)



Modular Neural Network



Radial Basis Function Network (RBFN)



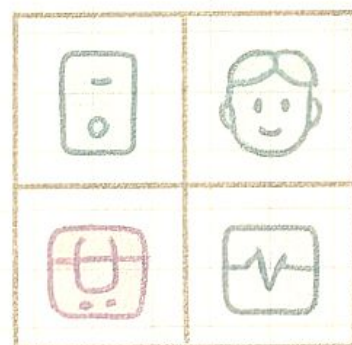
Made by Humna Imran

Applications of ANNs



Social Media:

- friend suggestions, recommendations, facial recognition



Marketing & Sales:

- personalized ads



Healthcare

- cancer detection
patient monitoring



Personal Vision:

voice recognition
NLP, generating answers



Personal Assistants

- voice recognition
NLP, generating answers



NLP = translation, sentiment



Computer Vision

- object detection
face recognition

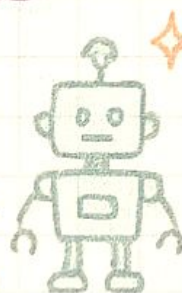


chatbots



The Future of Neural Networks

- Faster models (100x better)
- Giant models (108+ parameters)
- Huge economic value (\$15.7 trillion by 2025)
- Improved transparency + hybrid AI



Made by Humno Imran

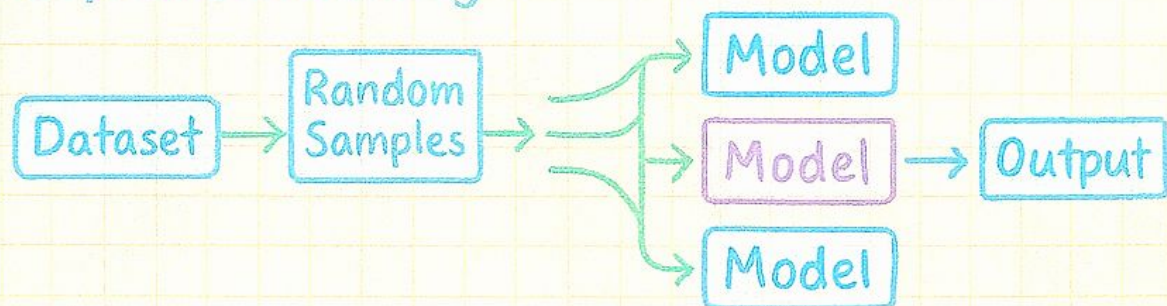


🌸 What is Bagging?

Bagging = Bootstrap Aggregating
Ensemble method

- Trains many models on random data subsets (sampled with replacement)
- Combines their outputs using voting (classification) or averaging (regression)

Goal: reduce variance, prevent overfitting, improve stability



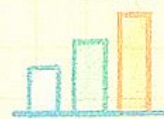
🕒 Steps in Bagging

- Prepare dataset
- Create bootstrapped samples
- Train base models on each
- Aggregate: majority vote / average
- Evaluate performance
- Tune with cross-validation



🧩 Ensemble Methods Overview

- Bagging: parallel training
- Boosting: sequential fixing
- Random Forest: bagging + trees + random features



MADE BY
HUMNA IMRAN

Why Bagging Works

- Reduces variance
- Prevents overfitting
- Improves generalization



Doodle idea: A stormy model calmed by ensemble voting icons

② Applications

- **Classification:** Medical diagnosis, churn prediction, spam detection
- **Regression:** Housing prices, stock forecasting
- **Anomaly Detection:** Fraud, network security
- **Feature Selection:** Highlights key-features
- **Imbalanced Data:** Handles class imbalance
- **Time-Series:** Forecasting, segmentation



Python Example (Sklearn)

- Use BaggingClassifier + DecisionTreeClassifier on Iris
- Evaluate using accuracy_score

✓ Conclusion:

Bagging = stable, accurate, and generalizable

- ✓ Great for noisy, complex data

Feature	Bagging	Boosting
Training Type	Parallel	Sequential
Focus	Variance	Blas
Sensitivity	Low	High
Bpeed	Faster	Slower
Examples	Random Forest	XGBoost, AdaBoost

MADE BY
HUMNA IMRAN

What Are Recommender Systems?

- Predict what user might like next
- Based on past behavior: clicks, purchases, views

🔦 Purpose:

- Help users choose in option-heavy environments
- Ex: Netflix → shows, Amazon → products, Spotify: playlist

🔦 Types:

- Collaborative Filtering (user-based & item-based)
- Content-Based Filtering (uses item features)
- Hybrid (combines both)

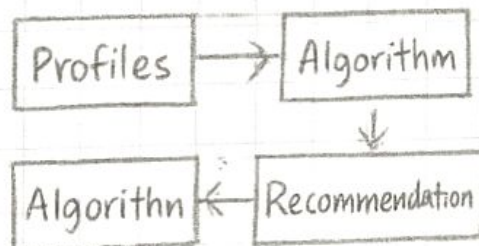
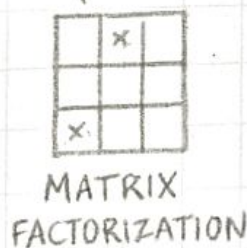
👍 Benefits:

- Speed decisions (ratings)
- Increase engagement
- Improves satisfaction
- Boosts retention

Business Impact	
Metric	Impact
Revenue	+35% suggestions
Netflix Views	+6% via rec system
CTR	+60% w/ personalization

⚠️ Advanced Techniques

- Cold start
- Filter bubble
- Data sparsity
- Scalability
- Privacy



🔦 Future

- Context-aware (mood, time)
- Explainable AI
- Federated learning
- Multimodal input (text, image, audio)

Power combo of math + ML + behavior
Drives Netflix, Amazon, YouTube
Moving toward smarter, safer systems

Drives Netflix, Amazon, YouTube
Moving toward smarter, safer systems

Netflix, Amazon, YouTube

What is Reinforcement Learning

- Agent interacts with environment
- Learns from feedback: rewards & penalties
- Goal: maximize cumulative reward over time

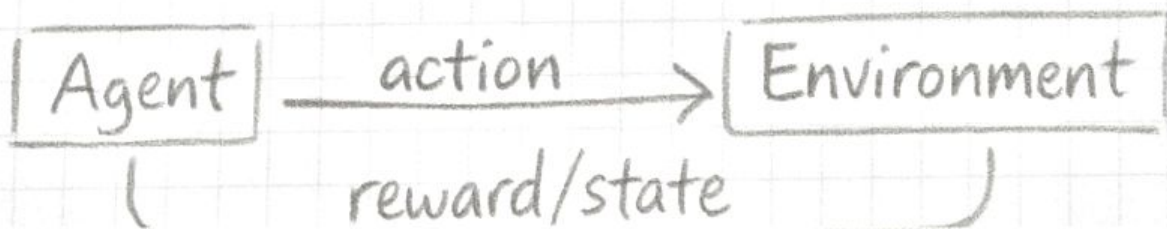
RL vs Supervised Learning:

- No labeled data
- Learns through trial & error
- Exploration vs. Exploitation

Key Building Blocks:

Term	Example
Agent	Robot in warehouse
Environment	Warehouse layout
State	Robot's position
Action	Move, turn, pick item
Reward	+10 delivery, -5 wall hit

RL Process (Diagram):



Key Mathematical Concepts:

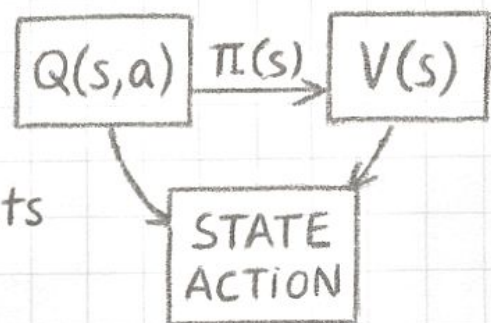
- Policy $\pi(s)$: action to take in state s
- Reward Function $R(s, a)$: value of action in a
- Value Function $V(s)$: future total reward
- Q-Value $Q(s, a)$: reward expected from action a in state s

Important Algorithms:

- Q-Learning $\rightarrow$ off-policy, learns best $Q(s, a)$
- DQN $\rightarrow$ Q-learning + neural nets
- Policy Gradient $\rightarrow$ directly optimize policy
- Actor-Critic $\rightarrow$ uses both policy & value

Applications of RL:

-  Robotics: walk, grasp
-  Gaming: AlphaGo, Dota bots
-  Autonomous Driving
-  Finance: trading bots
-  Healthcare: treatments
-  Smart Grid
-  Recommendations (YouTube, Netflix)



 Challenges: Exploration vs exploitation
Delayed rewards

 Summary: Deep environments = more compute

RL = agent learning from interaction

Rewards = feedback

Powerful for real-world, complex decision problems

Exploration vs. Exploitation

- Exploitation: choose known high-reward actions (e.g, repeat best chess move)
- Exploration: try unknown actions to discover better ones

Goal: balance for long-term reward, avoid local optima or wasted effort

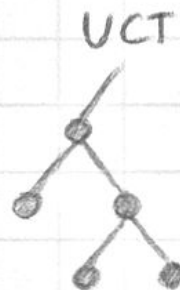
Monte Carlo Inference:

- Monte Carlo Methods
- Reinforcement Learning
- Bandit Problems
- Game AI, Robotics



Monte Carlo Tree Search (MCTS)

- Selection - follow best UCT path
- Expansion - add child nodes
- Simulation - rollout randomly
- Backpropagation - update rewards



UCT Score Formula:

$$\text{Score} = \bar{X}_j + c \cdot \sqrt{\frac{\ln N}{n_j}}$$

$\bar{X}_j$: avg reward (exploitation)

$c \cdot \sqrt{\dots}$: exploration bonus

Algorithms:

- UCT (used in MCTS)
- Epsilon-Greedy: ϵ chance of random move
- Annealing: reduce unknowns over time

Game AI & Planning:

AlphaGo: MCTS + Neural Net

Too much exploitation
= predictable

Too much exploration
= weak

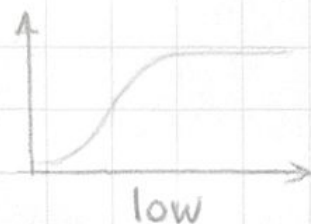
Planning: explore unknowns

🔥 Probabilistic Methods:

- Softmax / Boltzmann: randomize action choice by reward
High temp $\rightarrow$ more randomness
- Thompson Sampling: sample based on uncertainty
- Bayesian Optimization: pick based on value & uncertainty
- Contextual Bandits: explore based situation

🏠 Multi-Armed Bandits:

- Pick between unknown options Glot
Goal: maximize cumulative reward
- Used in A/B testing, ads, trials



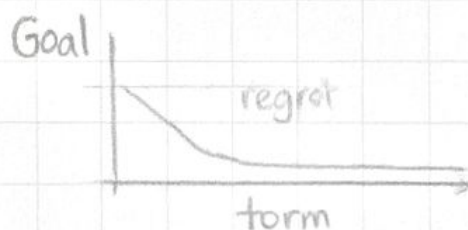
⚙️ Tuning Table:

- Convergence: all strategies
 $\rightarrow$ correct if explored enough
- Regret bounds
 $\rightarrow$ many algorithms grow regret slowly (e.g., $\log T$)

Tuning Table	
C	UCT exploration
E	Random choice rate
T	Softmax randomness
Prior	Initial belief weight domain-specific

🔗 Emerging Strategies:

- Meta-learning
- Info gain (entropy)
- Intrinsic curiosity
- Hierarchical exploration



📋 Final Takeaways:

- Balance is key
- Algorithms: UCT, ϵ -greedy, Thompson, Softmax
- Used in AlphaGo, robots, ads

📋 Final Takeaways:

- Balance is key
- Algorithms: UCT, ϵ -greedy, Thompson, Softmax
- Future = adaptive, efficient



What is a Cost Function?

- Measures how wrong the model is
- Ex: distance between prediction + actual

Goal: Minimize cost $\rightarrow$ better model

Most Common: Mean Squared Error (MSE)

$$J(\theta) = \frac{1}{2m} \sum (h_{\theta}(x) - y)^2$$

$h_{\theta}(x)$ = prediction, y = actual

m = # of examples



What is Gradient Descent?

Finds minimum of cost function

Updates parameters θ using slope

Formula: $\theta_j := \theta_j - \frac{\partial J(\theta)}{\partial \theta_j}$ α = learning rate
 ∂J = gradient



Learning Rate Table:

α Value
Slow convergence

α Value	Result
Too Small	Slow convergence
Too Big	Overshoots, unstable
Just Right	Fast, stable learning



Variants of Gradient Descent:

Batch GD: all data per step $\rightarrow$ stable, slow

SGD: 1 example per step $\rightarrow$ fast, noisy

Mini-Batch: small groups $\rightarrow$ balanced

Example: Learn $y = 2x$

$x = [1, 2, 3], y = [2, 4, 6]$

Init: $\theta_0 = 0, \theta_1 = 0, \alpha = 0.1$

→ After 1 iter. → $\theta_0 \approx 0.4, 9333$

→ Line: $h_0(x) \approx 0.4 + 0.9333x$

2 Gradient Descent Loop:

Predict → $h_0(x)$

Compute cost → $J(\theta)$

Compute gradients → $\partial J / \partial \theta$

Update params → $\theta_j: \theta_j := \theta_j - \alpha \cdot \nabla$

3 Advanced Optimizers:

Momentum

AdaGrad per-parameter learning

RMSProp better

Adam → adaptive

Cost Functions by Problem

Regression

MSE

Binary Classify

Binary Cross-Entropy

Multi-Class

Categorical Cross-Entropy

8 Tips:

Normalize features

Good θ init

Set stopping rules

Monitor cost graph

Python Demo:

After 100 iters: $\theta_0 \approx 0.1 = 2$

→ close to $y = 2x$

Final Takeaways:

Cost → how wrong

GD → corrects step by step

Regularize to generalize

3 Regularization

Penalty for complexity

$J_{\text{reg}}(\theta) = J(\theta) + \lambda \cdot R(\theta)$

L1: sum of θ

L2: sum of θ^2

Made by Humna